Formulaires HTML et PHP

Les **formulaires HTML** permettent de collecter des données utilisateur (nom, email, mot de passe, etc.), tandis que **PHP** permet de traiter ces données côté serveur. Cette interaction est fondamentale dans toute application web dynamique.

11.1 Méthodes GET et POST

Les données d'un formulaire sont envoyées au serveur via deux méthodes principales : **GET** et **POST**.

Méthode GET

- Les données sont envoyées dans l'URL.
- Visible dans la barre d'adresse.
- Limitée en taille.
- Moins sécurisée.

Exemple :

http://site.com/page.php?nom=Karim&age=25

Méthode POST

- Les données sont envoyées dans le corps de la requête.
- Non visibles dans l'URL.
- Plus sécurisée que GET.
- Permet d'envoyer de grandes quantités de données.

Comparaison :

Critère	GET	POST
Visibilité	Visible dans URL	Invisible
Sécurité	Faible	Meilleure
Taille	Limitée	Illimitée (pratique)
Usage	Recherche, filtres	Formulaires sensibles

11.2 Récupération des données

PHP utilise des **variables superglobales** pour récupérer les données :

- $_GET → données envoyées via GET

- $_POST → données envoyées via POST
- $_REQUEST → mélange des deux

Exemple complet :

Formulaire HTML :

```
<form method="post"
action="traitement.php">
    Nom : <input type="text"
name="nom">
    <input type="submit"
value="Envoyer">
</form>
```

Traitement PHP :

```
<?php
$nom = $_POST["nom"];
echo "Bonjour, " . $nom;
?>
```

11.3 Validation et filtrage

⚠️ **Étape essentielle en cybersécurité** : ne jamais faire confiance aux données utilisateur.

1. Vérification des données

```
if (isset($_POST["nom"]) &&
!empty($_POST["nom"])) {
    echo "Nom valide";
} else {
    echo "Champ obligatoire";
}
```

2. Nettoyage des données

- Évite les attaques comme **XSS (Cross-Site Scripting)**

```
$nom =
htmlspecialchars($_POST["nom"]);
```

3. Validation avec filter_var()

```
$email = $_POST["email"];

if (filter_var($email,
FILTER_VALIDATE_EMAIL)) {
    echo "Email valide";
} else {
    echo "Email invalide";
}
```

4. Bonnes pratiques de sécurité

- Toujours vérifier `isset()` et `!empty()`
- Utiliser `htmlspecialchars()` pour éviter XSS
- Utiliser `filter_var()` pour valider les données
- Ne jamais faire confiance aux entrées utilisateur
- Préférer la méthode **POST** pour les données sensibles

Exemple complet sécurisé

```
<?php
if ($_SERVER["REQUEST_METHOD"] ==
"POST") {
    $nom =
htmlspecialchars($_POST["nom"]);

   if (!empty($nom)) {
        echo "Bonjour, " . $nom;
    } else {
        echo "Veuillez entrer un
nom";
    }
}
?>
```

Résumé pratique

- Les formulaires permettent de **collecter des données utilisateur**.
- Méthodes principales :
  - **GET** (visible, limité)

    - **POST** (sécurisé, recommandé)
- Les données sont récupérées via `$_GET`, `$_POST`.
- Toujours **valider et filtrer** les données pour éviter les attaques.